

BLOC 3 — FRAMEWORK

Support d'oral (10 minutes) Ce qui est attendu par le jury et comment j'y réponds

Projet Airlines Reservation — Framework utilisé : Flask (Python).
Chaque point cite le fichier et la ligne de code correspondante,
pour pouvoir l'ouvrir en direct devant le jury si besoin.

MINUTAGE CONSEILLÉ (10 MIN)

0:00	Contexte : score du 1er passage (67/100) et point à corriger
1:00	C5.a — S'approprier l'architecture Flask via sa documentation
3:30	C5.b — Gestionnaire de dépendances et configuration
5:30	C5.c — Espace d'administration : CRUD complet (le point corrigé)
9:00	Conclusion + limites assumées



Commentaire du jury

67/100

« Le framework est compris et bien pris en main. Le projet est fonctionnel, mais pourrait être davantage approfondi, notamment avec la mise en place d'un espace d'administration et de fonctionnalités complètes de gestion des données : ajout, modification (UPDATE) et suppression (DELETE). »



Ce que ça donnait déjà

C'était le bloc le mieux noté du premier passage : les 3 compétences (C5.a, C5.b, C5.c) étaient acquises à 2/3 chacune. Le seul manque concret pointé par le jury — l'absence d'espace d'administration avec un vrai CRUD — est traité en détail dans la compétence C5.c ci-dessous.

À dire : « Ce bloc était déjà mon meilleur score au premier passage. Le jury a pointé un manque précis — pas d'espace d'admin avec ajout/modification/suppression — et c'est exactement ce que j'ai construit depuis. »

C5.a S'approprier l'architecture du framework via sa documentation

ATTENDU DU JURY

Ne pas se contenter d'utiliser Flask de façon basique, mais combiner plusieurs mécanismes du framework en s'appuyant sur sa documentation officielle.

CE QUE J'AI FAIT

J'utilise trois extensions officielles de l'écosystème Flask, chacune configurée selon sa propre documentation : **Flask-Login** pour les sessions et l'authentification, **Flask-WTF** pour la protection CSRF, et **Flask-Limiter** pour limiter le nombre de requêtes. Pour aller plus loin, j'ai construit un décorateur maison, `admin_required`, qui combine le décorateur natif `@login_required` de Flask-Login avec ma propre vérification de rôle (`current_user.is_admin`) — il faut comprendre précisément comment Flask-Login gère `current_user` pour pouvoir l'étendre. J'ai aussi mis en place un gestionnaire d'erreur personnalisé qui intercepte le code HTTP 429 (trop de requêtes) généré automatiquement par Flask-Limiter, pour rediriger l'utilisateur avec un message clair au lieu de lui montrer la page d'erreur brute par défaut.

```
app.py : ligne 75 - def admin_required(f): ... @login_required ...
```

```
app.py : lignes 84 - if not current_user.is_admin:
```

```
app.py : ligne 37 - @app.errorhandler(429)
```

À dire : « Je ne me suis pas arrêté aux tutoriels de base : j'ai combiné trois extensions officielles et écrit mon propre décorateur en étendant le mécanisme d'authentification de Flask-Login, ce qui suppose d'en comprendre le fonctionnement interne. »

C5.b Configurer le framework avec un gestionnaire de dépendances

ATTENDU DU JURY

Que les dépendances du projet et la configuration sensible soient gérées proprement, pas codées en dur.

CE QUE J'AI FAIT

Toutes mes dépendances sont gérées par **pip** avec un fichier `requirements.txt`, et je fige les versions précisément (`Flask==3.1.3`, `Flask-Login==0.6.3`, etc.) pour garantir que l'environnement d'installation reste identique et compatible dans le temps. Toute la configuration sensible — connexion à la base de données, clé secrète de session, paramètres SMTP pour l'envoi d'email — est centralisée dans un fichier `.env`, jamais commité sur GitHub, et chargée au démarrage via **python-dotenv**. Cela sépare clairement la configuration sensible du code source : 14 variables d'environnement au total.

```
requirements.txt : lignes 6-9, 16, 18 - Flask==3.1.3 , Flask-Limiter==4.1.1 , Flask-Login==0.6.3 , Flask-WTF==1.3.0 , psycpg2-binary==2.9.12 , python-dotenv==1.2.2
```

```
Database/db.py : lignes 3 et 5 - from dotenv import load_dotenv / load_dotenv()
```

```
.env (non versionné, exclu via .gitignore) - 14 variables : DATABASE_URL, SECRET_KEY, DB_HOST/NAME/USER/PASSWORD, SMTP_HOST/PORT/USER/PASS/SENDER_EMAIL/SENDER_NAME, SUPPORT_EMAIL, WEBSITE_URL
```

À dire : « Les versions de mes dépendances sont figées dans `requirements.txt`, et toute la configuration sensible — base de données, clé secrète, SMTP — vit dans un fichier `.env` qui n'est jamais poussé sur GitHub. »

C5.c — Application évolutive : l'espace d'administration

≈ 3,5 min — point corrigé du jury

Ce qui est attendu par le jury

Exactement le point soulevé au premier passage : mettre en place un espace d'administration avec des fonctionnalités complètes de gestion des données (ajout, modification, suppression), et montrer une application capable d'évoluer, d'être débuggée et livrée fonctionnelle.

Ce que j'ai fait — un vrai CRUD, pas une démo

- Un espace d'administration **complet**, avec une mise en page séparée du site client (une sidebar dédiée, sans header ni footer commercial).
- Les 4 routes correspondantes : lister les comptes clients, en créer un nouveau, modifier un compte existant, en supprimer un.
- Un administrateur est **automatiquement redirigé** vers ce dashboard dès sa connexion, sans que cela change quoi que ce soit au parcours des clients classiques.
- Toutes ces routes sont protégées par le décorateur `admin_required` vu en C5.a — **vérifié avec de vraies requêtes HTTP**, en utilisant deux comptes distincts, pas seulement en relisant le code : un visiteur non connecté est redirigé vers la connexion, un client connecté mais non-admin reçoit une erreur 403.

app.py : ligne 91 – `GET /admin/comptes` (liste)

app.py : ligne 133 – `POST /admin/api/comptes` (création)

app.py : ligne 165 – `PUT /admin/api/comptes/<id>` (modification)

app.py : ligne 211 – `DELETE /admin/api/comptes/<id>` (suppression)

app.py : lignes 92, 134, 166 – `@admin_required` sur chaque route

app.py : lignes 473-474 – `if user['is_admin']: return redirect(url_for('admin_comptes'))`

Un vrai bug rencontré et corrigé pendant le développement

L'ordre des colonnes renvoyées par une requête SQL ne correspondait plus à l'ordre attendu par mon code Python, ce qui aurait pu faire apparaître n'importe quel utilisateur comme administrateur. Corrigé définitivement en passant à un curseur qui renvoie des **dictionnaires nommés** plutôt que des tuples positionnels (`user['email']` plutôt que `user[2]`), pour éliminer ce risque d'erreur silencieuse — même si l'ordre des colonnes change plus tard en base.

app.py : ligne 11 – `from psycopg2.extras import RealDictCursor`

app.py : lignes 65, 95, 247 – `cursor_factory=RealDictCursor` – accès aux colonnes par nom

Robustesse : gestion d'erreurs systématique

L'application repose sur une gestion d'erreurs cohérente dans `app.py` : **20 blocs try/except** et **23 blocs finally**, qui garantissent qu'une connexion à la base est toujours fermée même en cas d'erreur, et que je ne montre jamais une trace technique brute au visiteur.

À dire : « J'ai répondu très précisément au manque signalé : un espace d'admin séparé, un CRUD complet sur les comptes clients, protégé par le même décorateur que celui présenté en C5.a, et vérifié avec de vraies requêtes HTTP »

sur deux comptes différents — pas juste en relisant le code. Au passage, j'ai trouvé et corrigé un vrai bug de correspondance de colonnes SQL qui aurait pu donner les droits admin à n'importe qui. »

✓ Ce qu'il faut retenir

- Le seul manque du premier passage — l'espace d'administration avec CRUD — a été traité intégralement : 4 routes, protection par décorateur, tests avec de vrais comptes.
- L'usage de Flask va au-delà du minimum : 3 extensions officielles combinées, décorateur maison qui étend Flask-Login, gestion d'erreur HTTP personnalisée.
- Configuration et dépendances proprement séparées du code (`.env` , `requirements.txt` à versions figées).
- Un vrai bug (ordre des colonnes SQL, risque de faille de droits) trouvé et corrigé pendant le développement — preuve de capacité à déboguer une application qui évolue.

⚠ Limite assumée (si le jury pose la question)

L'espace d'administration ne gère aujourd'hui que les comptes clients (pas les hôtels ou les vols) : le CRUD est complet sur ce périmètre, mais pourrait être étendu au catalogue si le temps le permettait.